

FreeRTOS基础（七）：临界段代码保护及调度器挂起与恢复

原创 VIP免费 于 2024-06-02 10:40:40 发布 · 2.4k 阅读 ·  30 ·  28

上一篇博客我们详细介绍了FreeRTOS是怎么管理中断的，其实，从本质上来讲就是将就是利用的BASEPRI这个寄存器，来屏蔽优先级低于某一断，当设置为0的时候，就是打开所有中断，所有中断都可以响应。这样就实现了对所有中断的管理！（中断的管理就是关闭某一个范围的中断和打开这一篇博客，我们介绍一个另外一个重要的概念，这在实际开发也有非常多的应用。

目录

- 一、临界段代码保护简介（熟悉）
 - 1.1 什么是临界段代码
 - 1.2 适用场合
 - 1.3 什么可以打断当前程序的运行（打断临界段）？
 - 1.3.1 中断
 - 1.3.2 任务调度
 - 1.4 如何避免打断临界段？
 - 1.5 临界段代码保护API函数介绍
 - 1.5.1 调用格式示例
 - 1.5.2 特点
 - 1.5.3 总结
- 二、任务调度器的挂起和恢复
 - 2.1 概念
 - 2.2 任务调度器的挂起和恢复API函数
 - 2.2.1 任务调度器挂起函数
 - 2.2.2 任务调度器恢复函数
 - 2.3 与使用关闭中断和打开中断实现临界段代码保护的区别
 - 2.4 挂起和恢复任务调度器适用场合
 - 2.5 使用格式示例
 - 2.6 任务挂起与恢复内部实现（了解）

一、临界段代码保护简介（熟悉）

1.1 什么是临界段代码

临界段代码也叫做临界区，是**指那些必须完整运行，不能被打断的代码段。**

1.2 适用场合

1, 外设	需严格按照时序初始化的外设：IIC、SPI等等
2, 系统	系统自身需求，操作系统源代码中有很多使用到
3, 用户	用户需求

CSDN @未来可期，静待花开~

1.3 什么可以打断当前程序的运行（打断临界段）？

1.3.1 中断

当中断发生的时候，当前运行的程序就会被暂停，转而去执行中断。

1.3.2 任务调度

当前任务正在运行，出现一个更高优先级的任务，那么当前运行的程序就会被阻塞住。

1.4 如何避免打断临界段？

通过上面，我们知道打断临界段的原因有哪些，那么我们就可以对症下药了，解决问题了。

首先，关闭中断，则中断就不会打断我的运行，其次，任务调度也打断不了我，因为：PendSV是实现任务调度的一个内核中断，并且这个中断被设置成最低15，这里的关闭中断指的是：在管理的中断优先级范围内（FreeRTOS所能管理的最高中断）的中断都不会响应，那么自然优先级PendSV中断也会被关闭，那么就无法进行任务切换调度，也就无法打断当前程序的运行！

1.5 临界段代码保护API函数介绍

通过上面的分析，我们就不谋而合了，FreeRTOS 在进入临界段代码的时候需要关闭中断（管理范围内的中断），当处理完临界段代码以后再打行正常的中断响应和任务调度。这就是临界段代码保护的方法！

函数	描述
<code>taskENTER_CRITICAL()</code>	任务级进入临界段（关中断）
<code>taskEXIT_CRITICAL()</code>	任务级退出临界段（开中断）
<code>taskENTER_CRITICAL_FROM_ISR()</code>	中断级进入临界段（关中断）
<code>taskEXIT_CRITICAL_FROM_ISR()</code>	中断级退出临界段（开中断）

CSDN @未来可期，静待花开

中断级的临界区函数会返回之前中断屏蔽寄存器的值，我们需要用一个变量接收，保存下来，然后关闭中断，退出临界区时将之前的中断屏蔽值写回去。

1.5.1 调用格式示例

任务级临界区调用格式示例：

```
taskENTER_CRITICAL();
{
    ... .. /* 临界区 */
}
taskEXIT_CRITICAL();
```

中断级临界区调用格式示例：

```
ISR (中断服务函数)
{
    uint32_t save_status;
    save_status = taskENTER_CRITICAL_FROM_ISR();
    {
        ... .. /* 临界区 */
    }
    taskEXIT_CRITICAL_FROM_ISR(save_status);
}
```

CSDN @未来可期，静

1.5.2 特点

- 1、成对使用。
- 2、支持嵌套。
- 3、尽量保持临界段耗时短。因为：在临界区屏蔽了中断，并且也不会发生任务调度了，中断一般是紧急的事，任务调度是可以满足实时性的，如长，可想而知，会带来什么后果！因此需要注意临界区代码的执行时间不宜过长，否则可能导致系统中断响应的延迟。

1.5.3 总结

临界段代码保护是直接屏蔽了中断（管理范围内），系统任务调度靠中断，ISR也靠中断，因此它的影响还是非常大的！！！！

二、任务调度器的挂起和恢复

2.1 概念

挂起就是暂停的意思，任务调度器挂起就是：任务不能再进行切换/调度，恢复就是任务调度器可以重新进行任务的切换。仅仅针对的是任务调度是否可以发生任务切换！

2.2 任务调度器的挂起和恢复API函数

函数	描述
vTaskSuspendAll()	挂起任务调度器
xTaskResumeAll()	恢复任务调度器

CSDN @未来可期，静待花开~

2.2.1 任务调度器挂起函数

- `vTaskSuspendAll()` 函数用于暂停任务调度器。调用这个函数后，任务切换将不会发生，系统会保持当前正在运行的任务。**挂起调度器不会关中断服务程序（ISR）仍然可以运行。**
- 这个函数**通常用于临界区代码保护，确保一段代码在执行过程中不会被任务切换打断。**

2.2.2 任务调度器恢复函数

- `xTaskResumeAll()` 函数用于恢复任务调度器。恢复后，如果有较高优先级的任务被挂起期间解锁，那么这些任务将会立即执行。如果没有优先的任务，当前任务将继续执行。
- 在恢复任务调度时，系统会检查任务是否需要上下文切换，以确保高优先级任务能及时执行。

2.3 与使用关闭中断和打开中断实现临界段代码保护的区别

FreeRTOS提供了两种主要的临界段代码保护方法：

关闭和打开中断（`portENTER_CRITICAL()` 和 `portEXIT_CRITICAL()`）：

- `portENTER_CRITICAL()` 函数**关闭中断，确保在临界区代码执行期间不会被管理范围内的中断打断。也不会发生任务的切换。**
- `portEXIT_CRITICAL()` 函数重新打开中断，允许系统恢复正常中断处理。
- 优点：
 - 提供了最高级别的临界区保护，因为它**完全阻止了中断。**
 - **适用于非常短且时间敏感的代码段。**
- 缺点：
 - 关闭中断会影响整个系统的中断响应时间，可能会导致高优先级中断的延迟处理。
 - 不适用于长时间执行的代码，因为这会严重影响系统的实时性。

挂起和恢复任务调度器（`vTaskSuspendAll` 和 `xTaskResumeAll`）：

- `vTaskSuspendAll()` **暂停任务调度，但不会影响中断。这意味着ISR仍然可以执行，且ISR可以发起上下文切换（如果在ISR中调用了Yield函数）。**
- `xTaskResumeAll()` 恢复任务调度，并在必要时进行上下文切换。
- **它仅仅是防止了任务之间的资源争夺，中断照样可以直接响应；中断可以抢占当前任务的资源，但是其他任务无法抢占当前任务资源**
- 优点：
 - **保持中断响应能力，因为中断不会被禁用。**
 - 适用于需要保护不被任务切换打断的长时间执行代码，但又不需要完全禁止中断的场景。
- 缺点：
 - 不适用于极端时间敏感的代码保护，因为ISR仍然可以打断执行。
 - 任务调度恢复时需要检查并可能进行任务切换，增加了一些开销。

总结一下就是：**利用任务调度器的挂起和恢复实现的临界段代码保护只是不会发生任务的切换，但是中断并未关闭，在临界区，中断依然可以响应，用关中断和开中断实现的临界段代码，它直接关闭了管理范围内的中断，任务切换也不会发生！它的力度是非常强的！**

2.4 挂起和恢复任务调度器适用场合

挂起调度器的方式，适用于临界区位于任务与任务之间；既不用去延时中断，又可以做到临界区的安全。就是说：它不会受到任务切换而被打断，保证中断的及时响应！

1. 任务与任务之间的临界区：

- 1. 在多任务操作系统中，多个任务之间可能需要访问共享资源（例如全局变量、硬件设备等）。为了防止任务之间的资源竞争，需要保持操作。
- 2. 临界区是指访问共享资源的代码段，需要确保这个代码段在一个任务执行时不会被其他任务打断。

2. 不延时中断：

- 1. 挂起任务调度器 (vTaskSuspendAll()) 的方式不会关闭中断，这意味着系统中的中断服务程序 (ISR) 仍然可以正常执行。ISR的执行不受挂起任务调度器的影响。
- 2. 由于中断没有被关闭，系统的中断响应时间不会受到影响。这样可以保证系统的实时性，因为时间关键的中断仍然能够及时处理。

3. 临界区的安全：

- 1. 当任务调度器被挂起时，当前任务可以独占CPU资源执行临界区代码，而不必担心在这个过程中会被其他任务打断。这样可以确保临界区的代码安全执行。
- 2. 任务调度器挂起期间，虽然ISR可以执行，但ISR不会引起任务切换。因此，任务执行临界区代码的过程中，不会被其他任务打断。

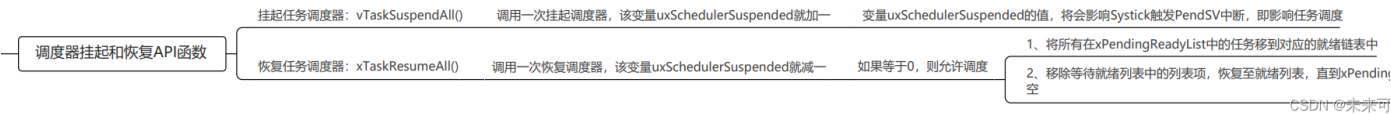
2.5 使用格式示例

使用格式示例：

```
vTaskSuspendAll();  
//任务不会进行切换，但是内部的代码可以被中断打断！  
{  
    ... ..    /* 内容 */  
}  
xTaskResumeAll();
```

CSDN @未来可期，静待花开~

2.6 任务挂起与恢复内部实现（了解）



至此，已经讲解完毕！初次学习，循序渐进，一步步掌握即可！以上就是全部内容！请务必掌握，创作不易，欢迎大家点赞加关注评论，您的支持是的动力！下期再见！

标签 #单片机 #FreeRTOS

